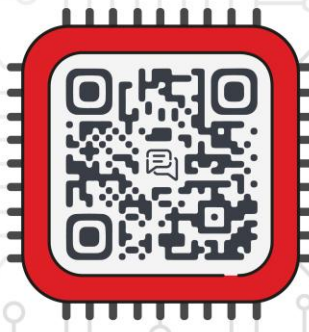




1



17



2040 sp

عملي

كلية الهندسة المعلوماتية

السنة الثانية

Introduction to tree concepts

د. عملي مشترك



RB Informatics; 16/04/2025

محتوى مجاني غير مخصص للبيع التجاري

خوارزميات ٢

في المقرر السابق للخوارزميات تحدثنا بصورة عامة عن بنى المعطيات الخطية (stack-queue-LinkedList-array) و بعض الخوارزميات التي يمكن تطبيقها عليها (searching / sorting) و بعض الخوارزميات العامة المستخدمة في حل المشاكل:

(divide and conquer/brute force/DP/backtrack)

في هذا الفصل في مقرر الخوارزميات 2 سنتعرف على الـ TREES والـ GRAPH و هما بنيتا معطيات غير خطيتين و بعض الخوارزميات التي يمكن تطبيقها عليهما

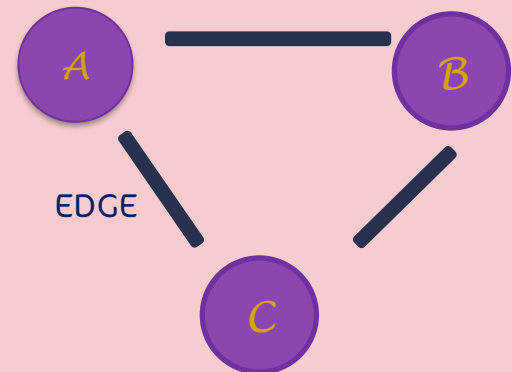
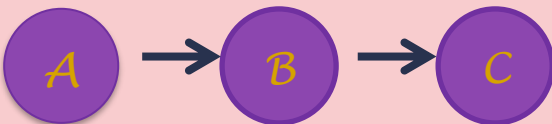
سنقوم بالتمهيد لموضوع محاضرتنا من خلال معلومات نظرية بإطار الـ GRAPH THEORY

What is graph?

بشكل مجرد يمكن توصيف الـ graph بأنه مجموعة من العقد (nodes) التي ترتبط فيما بينها بوصلات (edges) يمكننا من خلاله تمثيل مجمل العلاقات غير الخطية بين البيانات مثلاً: (الطرق في الخرائط – شبكات التواصل الاجتماعي – علاقات المجلات ببعضها البعض) ، و أيضاً يمكن استخدامها لتمثيل علاقات خطية كما سبق وفعلنا بالـ linked list (يمكن اعتبار linked list حالة خاصة من الـ graph)

NODE

Linked list

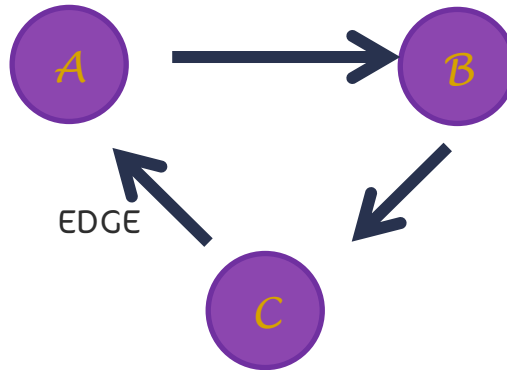


يمكن تصنيف الـ GRAPHS من ناحية نوع الـ EDGE فيكون له نوعان :

1. **DIRECTED GRAPH** : حيث يمكن للـ EDGE أن تكون مساراً بين العقدة A والعقدة B وهذا المسار موجه بجهة واحدة محددة فقط .

(ويعني ذلك يمكن الذهاب من العقدة A إلى العقدة B ولكن العكس غير صحيح من خلال هذا المسار)

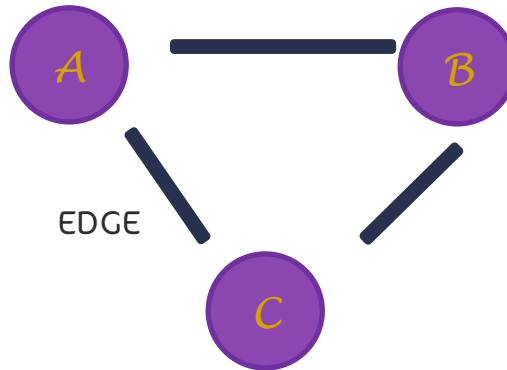
NODE

DIRECTED GRAPH

2. **UNDIRECTED GRAPH** : حيث يمكن للـ EDGE أن تكون مساراً بين العقدة A والعقدة B وهذا المسار غير موجه لجهة محددة.

(ويعني يمكن الذهاب من العقدة A إلى العقدة B والعكس صحيح من خلال هذا المسار).

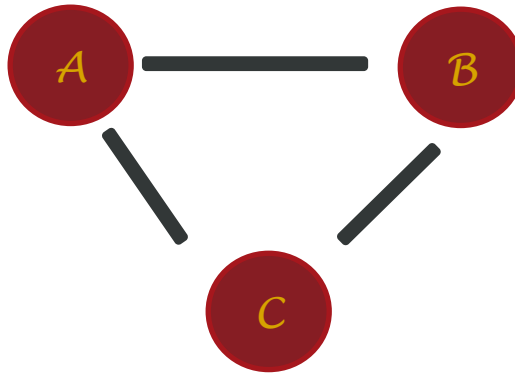
NODE

You can use this in
both directionsUNDIRECTED GRAPH

❧ و يمكن تصنيفه من ناحية وجود الـ cycles فيكون له نوعان :

1. **cyclic** : المرور بمسار ما ضمن هذه العقد يؤدي للعودة لعقدة قد زرتها مسبقاً
 $a \rightarrow b \rightarrow c \rightarrow b \rightarrow c \dots / a \rightarrow b \rightarrow c \rightarrow a \rightarrow b \dots$
 و بالتالي تصبح في دائرة مفرغة و المسار سيوصلك لنقطة قد وصلنا لها مسبقاً

CYCLIC GRAPH



2. **acyclic** : المرور بمسار ما في هذه العقد حتماً لن يؤدي للعودة لنقطة قد زرتها مسبقاً $a \rightarrow b \rightarrow c$ بالتالي كل العقد فريدة بالمسار الواحد و سنصل دائماً لعقد جديدة حتى ينتهي المسار

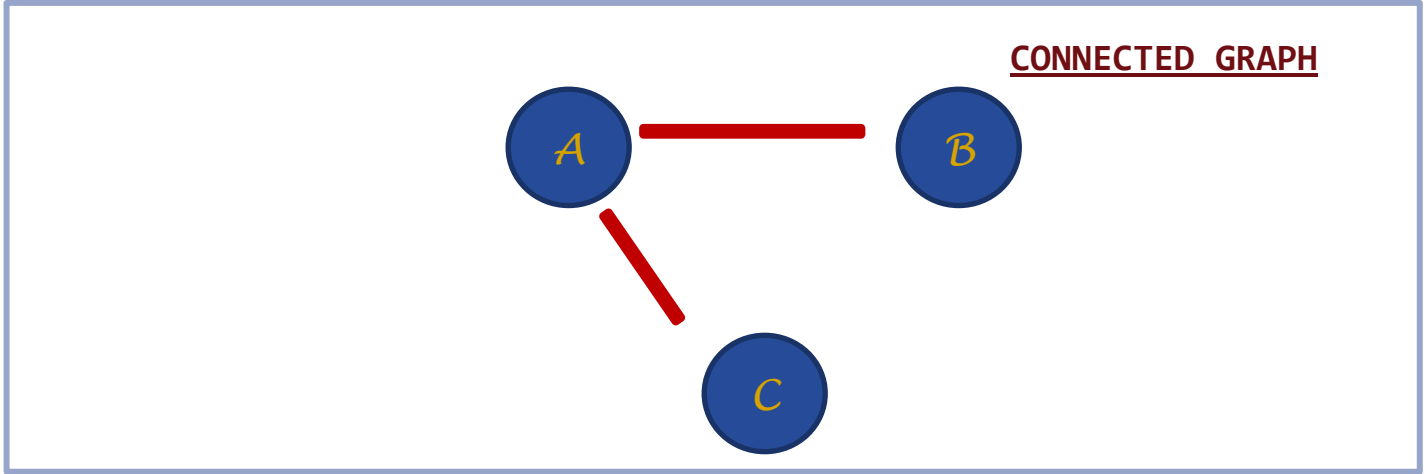
ACYCLIC GRAPH



❧ و يمكن تصنيفه من ناحية وجود قيمة لكل EDGE فيكون له نوعان:

1. **WEIGHTED GRAPH** : حيث كل EDGE لها قيمة معينة (وزن)
 (يمكن استخدامه أثناء التعبير عن خريطة جغرافية حيث لدينا مدن يفصل بين كل مدينة و مدينة مسافة محددة ، فكل مدينة تكون عقدة و كل EDGE تصل بين عقدتين تحمل قيمة تعبر عن المسافة بين المدينتين)
2. **UNWEIGHTED GRAPH** : حيث ليس هنالك قيمة لأي EDGE
 (يمكن استخدامه أثناء التعبير عن الأصدقاء في موقع من مواقع التواصل الاجتماعي حيث كل مستخدم عقدة و يصل بينه و بين أصدقاءه EDGES و لا حاجة لامتلاك هذه الـ EDGES لأي قيمة)

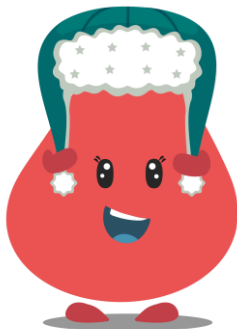
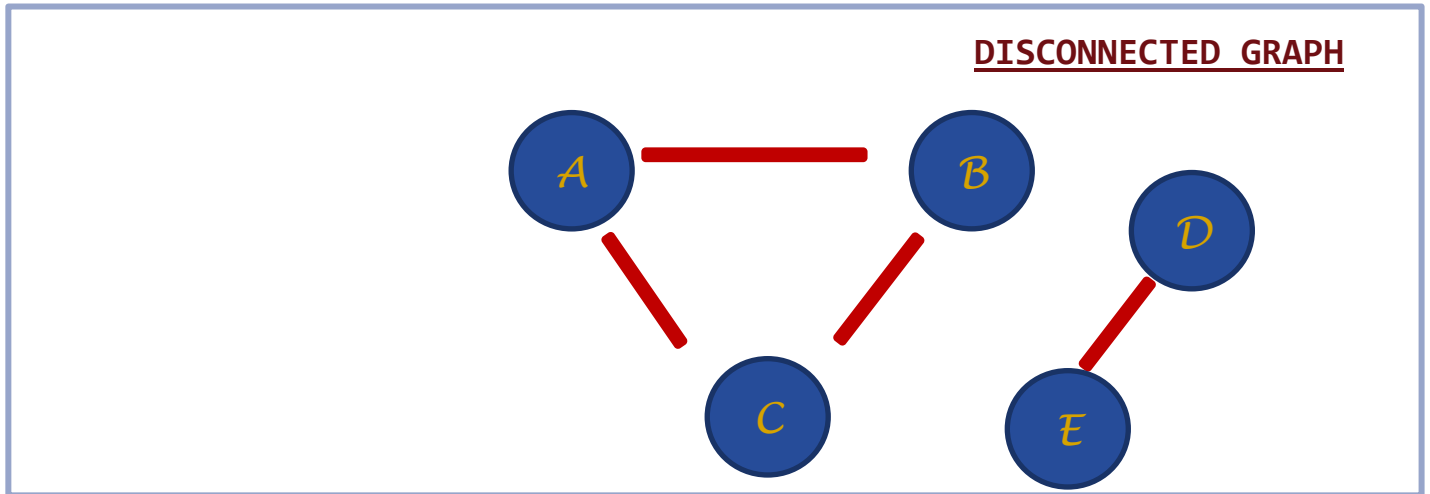
و يمكن تصنيفه من ناحية اتصال عقد هذا ال graph بعضها فيكون له نوعان:
1. **CONNECTED**: كل العقد تتصل بعضها بطريقة مباشرة أو غير مباشرة .



2. **DISCONNECTED**: هنالك عقدتان على الأقل لا يوجد بينهما اتصال بطريقة مباشرة أو غير مباشرة

ملاحظة:

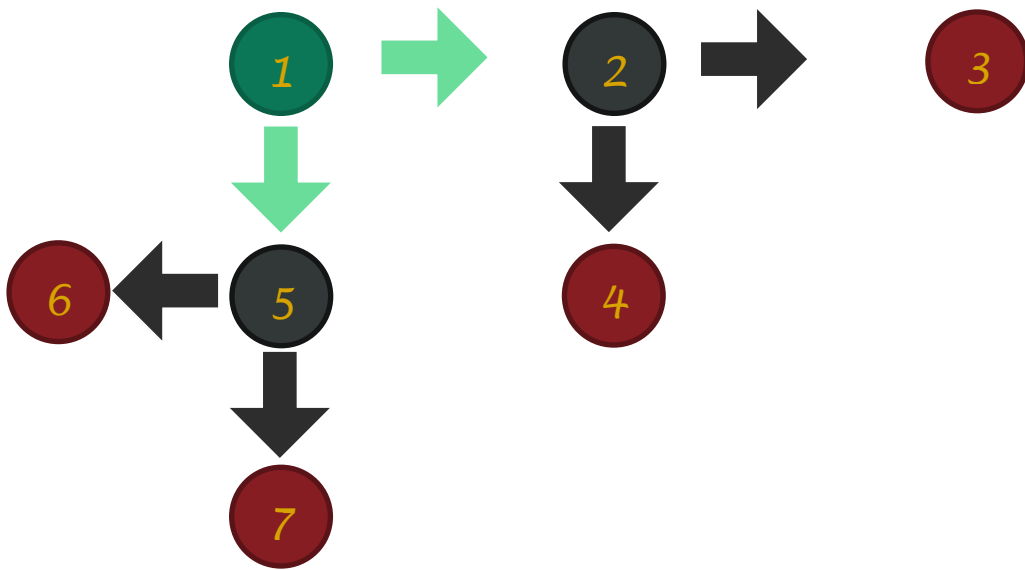
(في ال DIRECTED GRAPH هنالك نوعان من ال CONNECTIVITY و لن نناقشها هنا).



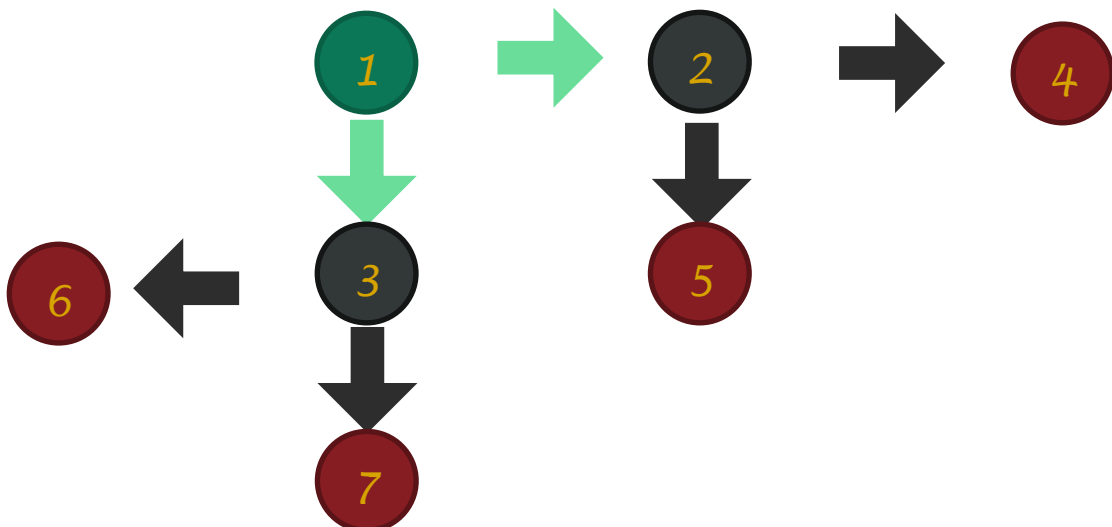
GRAPH TRAVERSAL

بشكل عام حتى نقوم بالمرور على العناصر الموجودة و المخزنة ضمن الـ GRAPH
لا بد لنا من انتهاج أحد الطريقتين التاليتين:

1. DFS : و الذي يعتمد على فكرة أنه بالنسبة لكل عقدة أقف عندها عليّ المضي بمسار ما حتى الوصول لأعمق نقطة فيه ، ثم العودة و تجربة مسار آخر حتى نهاية كل مسارات العقدة.

DFS TRAVERSAL

2. BFS : يعتمد على فكرة أنه بالنسبة لكل عقدة أقف عندها عليّ تفقد كل العقد المجاورة لها ، ثم الانتقال للمستوى التالي من العقد.

BFS TRAVERSAL

كلا الطريقتين يمكن استخدامهما ، و نفاضل بينهما بناء على السياق المطلوب منا .

كانت هذه مقدمة كافية للمفاهيم المهمة ضمن الـ GRAPH

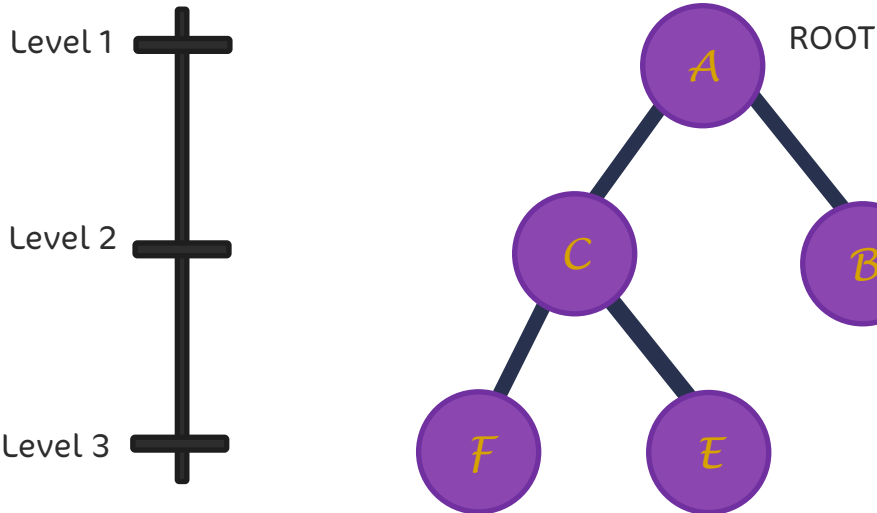
WHAT IS A TREE ??

لو عرفنا الـ TREE بإطار الـ GRAPH THEORY فهي باختصار CONNECTED ACYCLIC UNDIRECTED GRAPH و لكن نحن بإطار مقررنا سنتعرف بشكل أعمق على ما نسميه ROOTED TREE أي سنقف عند هذا الـ GRAPH المدقق للشروط ، و نختار له جذراً أو نقطة بداية و نتجه من هذه النقطة نحو باقي العناصر بحيث تصبح العلاقة بين العقد علاقة (أب-ابن) بدءاً من الجذر الذي اخترناه نزولاً نحو الفروع (يصبح لدينا اتجاه ضمني لهذه الشجرة "من الجذر و حتى الفروع").

عندما نتكلم بالـ TREES بصيغة عامة ضمن مقررنا سنقصد بها الـ ROOTED TREES و التي سنستخدمها كـ data structure لتمثل العلاقات الهرمية.

✎ و الآن سنتعرف على بعض المفاهيم في الـ ROOTED TREES :

- ROOT : نقطة البداية التي سنبدأ منها التعامل مع الشجرة بشكل دائم هذه العقدة ليس لها أب (لا يسبقها أي عقدة).
- تكون العقدة A أب للعقدة B إذا كان كلا العقدتين متصلتين و كانت A الأقرب للجذر.
- تكون العقدة A ابن للعقدة B إذا كانت كلا العقدتين متصلتين و كانت A الأبعد عن الجذر.
- تكون العقدة داخلية إذا كان لها ابن واحد على الأقل.
- تكون العقدة خارجية (leaf) إذا لم يكن لها أبناء.
- تكون العقدة A (ANCESTOR) للعقدة B إذا كانت العقدة A من العقد الواقعة في المسار من العقدة B و حتى الجذر (السلف).
- تكون العقدة A (DESCENDANT) للعقدة B إذا كانت العقدة B من العقد الواقعة في المسار من العقدة A حتى الجذر (السلالة).
- يمكن تسمية عدد طبقات الشجرة بمستوى الشجرة "TREE LEVEL".
- يمكن تسمية عدد أبناء العقدة بدرجة العقدة.



- هذه شجرة من المستوى الثالث.
- الجذر هو A العقد الداخلية B و C و العقد الخارجية F E D
- العقدة C هي أب للعقدة E و العقدتان E و F هما أبناء C
- العقدة F DESCENDANT للعقدة A والعقدة B والعقدة A هما ANCESTORS للنقاط F و E و D

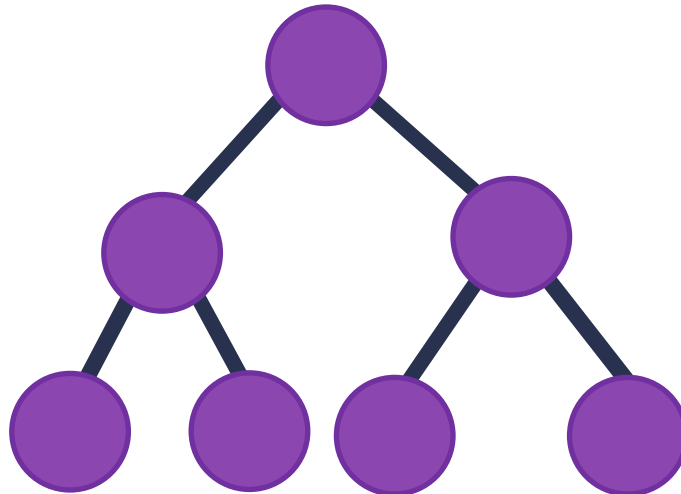
BINARY TREE

الآن سنتحدث عن نوع محدد أكثر من الأشجار و تمثيله برمجياً سنتحدث عن الـ BINARY TREE حيث سنقوم بوضع قيد إضافي و هو : درجة العقدة على الأكثر 2 (يمكن لكل عقدة أن تمتلك ابنين على الأكثر).

بعض أنواع الـ BINARY TREE :

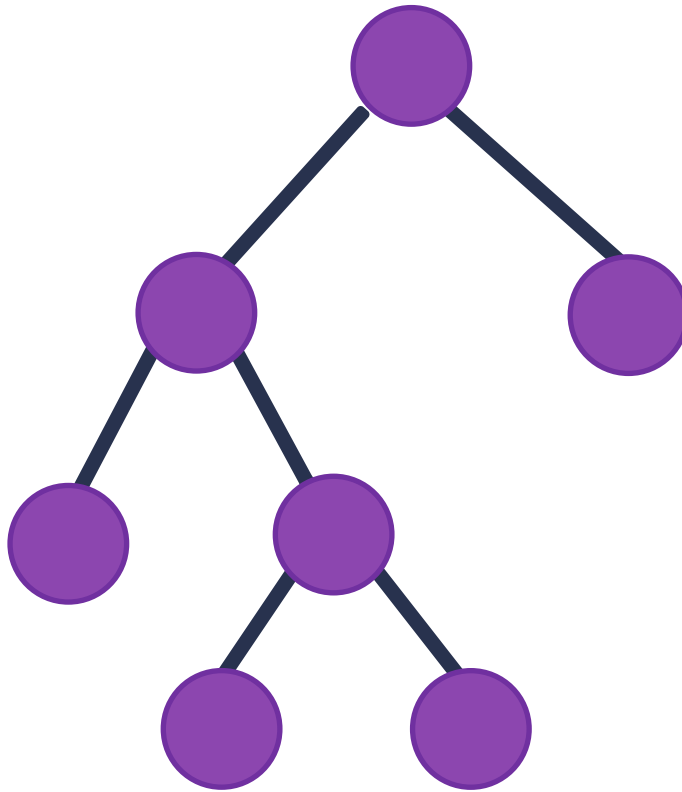
1. PERFECT BINARY TREE :

يجب لكل عقدة ضمن الشجرة أن تمتلك ابنين و كل العقد الخارجية يجب أن تقع بنفس المستوى



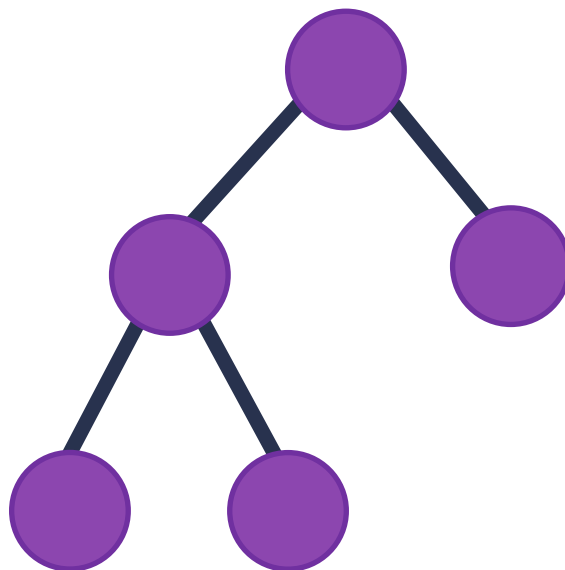
2. FULL BINARY TREE

كل عقدة يجب أن تمتلك ابنيين أو لا تمتلك أبناء بتاتاً.



3. COMPLETE BINARY TREE

يجب لكل عقدة أن تمتلك ابنيين إلا بالمستوى الأخير لا بأس بالنقص لكن النقص من طرف اليمين حصراً.



يوجد عدة طرق أهمها:

array based representation

بحيث جذر الشجرة هو العنصر في الـ index 1

و يكون for each element at the i index

the left child lies on $2i$ right child lies on $2i+1$

و حتى إن كان أحد الأبناء فرضاً غير موجود نترك مكانه كقيمة فارغة مثلاً: يمكن البدء بالجذر 0 at index

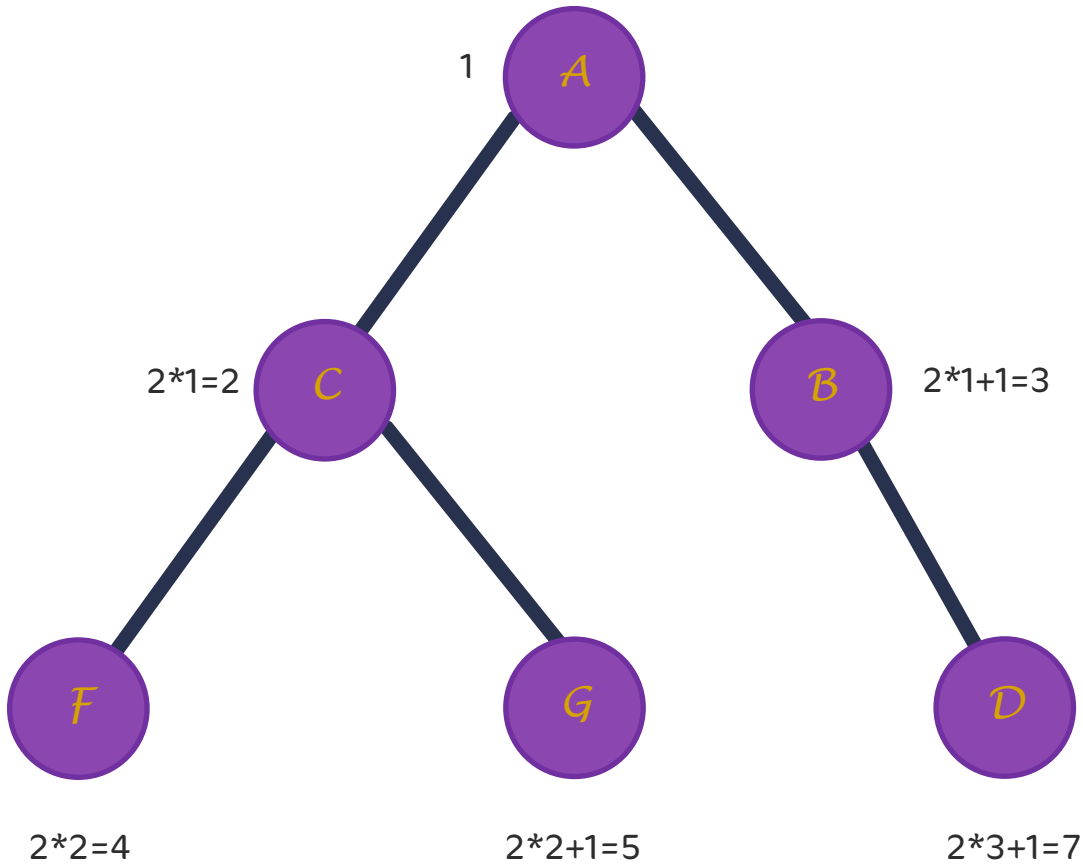
فيكون for each element at the i index

the left child lies on $2i+1$ the right child lies on $2i+2$

لو أردنا رسم هذه الشجرة حيث الجذر سيكون في

الـ index 1 ستكون هي كالتالي:

	a	c	b	f	g		d
0	1	2	3	4	5	6	7



TREE TRAVERSAL

بصورة عامة ذكرنا أهم عنصرين في الـ GRAPH TRAVERSAL : (DFS,BFS)، وفي الـ TREE لدينا نسق مشابه

- **DFS** : يعتمد على الابتداء بعقدة ثم الذهاب لأعمق مستوى ثم العودة للعقدة و تجربة مسار آخر في الأشجار ينقسم الـ DFS لثلاثة أنواع أساسية بناءً على ترتيب التفتد للعناصر ضمن الشجرة سنشرحها بطريقة **RECURSIVE** حتى يسهل تصورها.

ليكن لدينا شجرة صغيرة فيها عقدة X ولتكن جذراً ولها ابن يساري A وابن يميني B وبثلاثة طرق سيكون العبور وفق التالي

PREORDER Traversal

سيتمتد العقدة X ثم يذهب لليسار A ثم يذهب لليمين B فلو فرضاً كنا نطبع قيم سيطبع XAB

INORDER Traversal

سيتمتد الابن اليساري A ثم يتمتد العقدة X ثم يتمتد الابن اليميني B فلو طبعنا AXB

POSTORDER Traversal

سيتمتد الابن اليساري A ثم الابن اليميني B ثم يعود لـ X فلو طبعنا ABX

هذا على مستوى شجرة صغيرة فيها جذر وابنان فقط ويمكن تعميم هذا النمط على الأشجار الأكبر فنقوم باستدعاء عودي على الأبناء حتى نصل إلى الأوراق ثم نعود أدراجنا (backtrack) ونعمم أحد الأنماط السابقة.

للتوضيح أكثر: عند كل عقدة نقف عندها نعتبر العقدة الابن سواء اليميني أو اليسرى شجرة مستقلة ونستمر بالاستدعاء العودي عليها حتى الوصول إلى عقدة لا أبناء لها ومن هنا نوقف التابع العودي ونعود بالاستدعاءات حتى نصل لجذر الشجرة الأصلية.

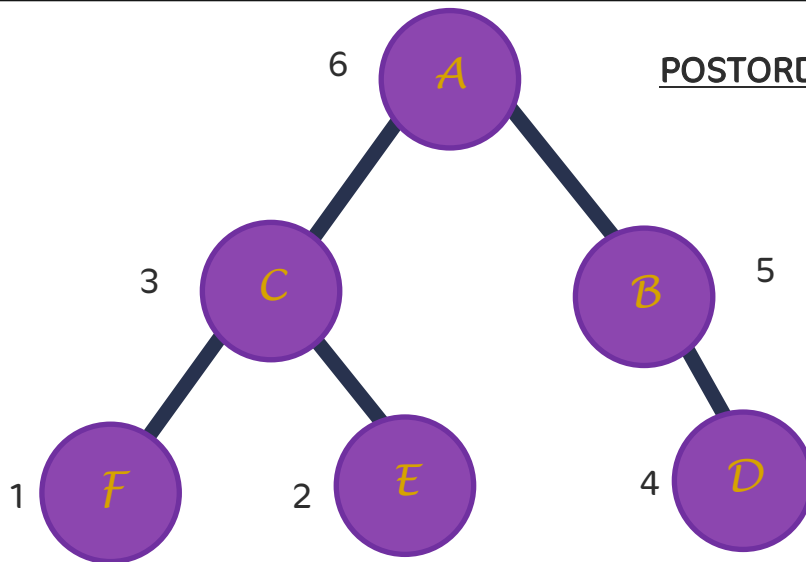
لماذا يمكننا تعريف التابع بشكل عودي؟

يمكن اعتبار كل عقدة نقف عندها جذر لشجرة أصغر من الشجرة الأصلية حتى نصل للأوراق (العقد الخارجية) التي يمكن اعتبارها أشجار فيها الجذر null أي شجرة لا عقد فيها (يمكن أن نعتبر هذا basecase) ثم نعود أدراجنا بالاستدعاءات حتى نصل إلى الجذر الأصلي للشجرة الكبرى التي بدأنا بها، وبهذا سنقوم بإعطاء مثال على ترتيب الطباعة على شجرة أعقد قليلاً لعل الفكرة تصبح أوضح ثم سنقوم بالتعريف على كيف يمكن كتابة الكود الخاص بكل نوع.

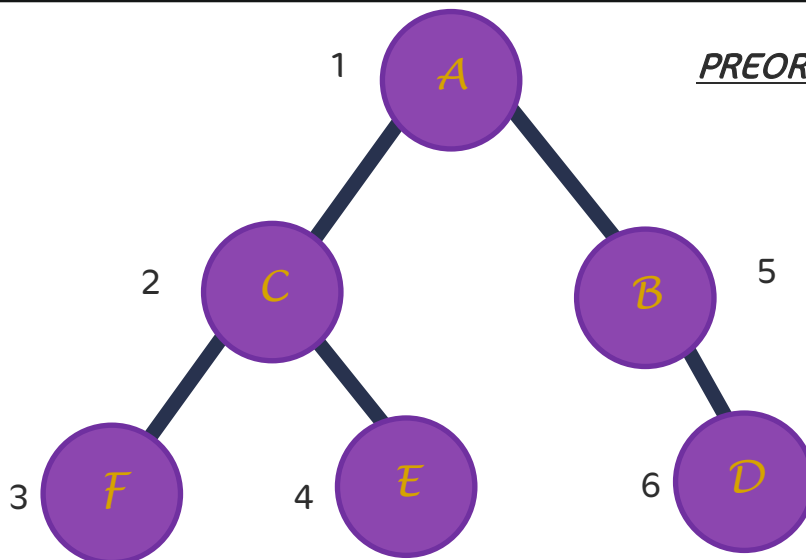
هل من غير الممكن القيام بهذا باستخدام نمط تكراري؟

بلى ، من الممكن لكن في الـ DFS من الأسهل دوماً استخدام التابع العودي كونه أكثر وضوحاً بالنسبة للقارئ سنعرض طريقة تكرارية لأحد الأنواع في نهاية المحاضرة.

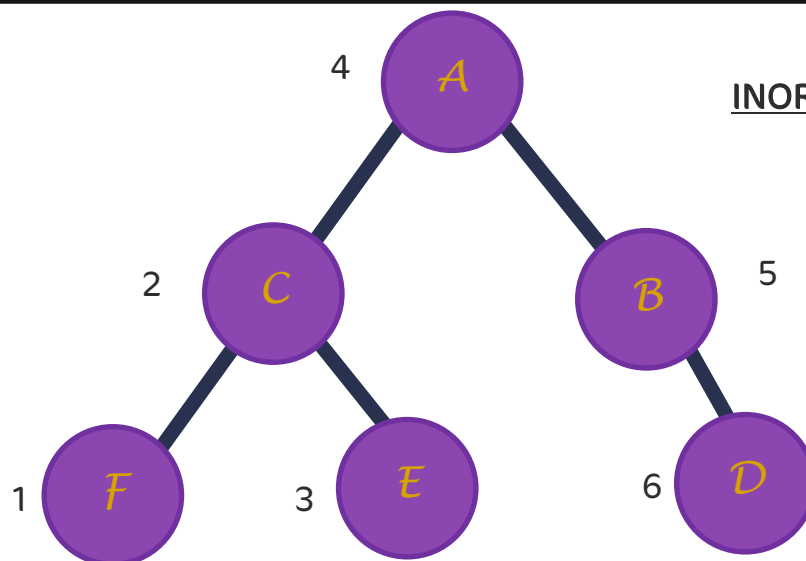
POSTORDER TRAVERSAL



PREORDER TRAVERSAL



INORDER TRAVERSAL



CODE

```
class BinaryTree{
    // المصفوفة التي تحمل عقد هذه الشجرة
    char []arr ;
    // باني نمرر له المصفوفة التي تمثل الشجرة
    BinaryTree(char []arr){
        this.arr = arr;}
    public void preOrderTraversal (int i){
        // شرط التوقف : إذا وجدت المكان فارغاً "null node" أو خرجنا خارج إطار المصفوفة قم بالتوقف
        if (i >= arr.length||arr[i]==' ')
            return ;
        // نطبع الجذر قبل الانطلاق إلى الأبناء
        System.out.print(arr[i]);
        // استدعاء التابع العودي على الابن الأيسر
        preOrderTraversal(2*i);
        // استدعاء التابع العودي على الابن الأيمن
        preOrderTraversal(2*i+1);
    }
    public void postOrderTraversal (int i){
        // شرط التوقف : إذا وجدت المكان فارغاً "null node" أو خرجنا خارج إطار المصفوفة قم بالتوقف
        if (i >= arr.length||arr[i]==' ')
            return ;
        // استدعاء التابع العودي على الابن الأيسر
        postOrderTraversal(2*i);
        // استدعاء التابع العودي على الابن الأيمن
        postOrderTraversal(2*i+1);
        // بعد الانتهاء من الاستدعاءات على كلا الابنين نطبع الجذر
        System.out.print(arr[i]);
    }
}
```

```

public void inorderTraversal (int i){
    // شرط التوقف :إذا وجدت المكان فارغاً "null node" أو خرجنا خارج إطار المصفوفة قم بالتوقف
    if (i >= arr.length||arr[i]==' ')
        return ;
    // استدعاء التابع العودي على الابن الأيسر
    inorderTraversal(2*i);
    // طباعة الجذر بعد انتهاء الاستدعاء العودي على الابن الأيسر
    System.out.print(arr[i]);
    // استدعاء التابع العودي على الابن الأيمن
    inorderTraversal(2*i+1);
}
}

public class Main {
    public static void main(String[] args) {
        // تمثيل للشجرة التي تكلمنا عنها ضمن مصفوفة سنمثل العقد الغير موجودة بمحرف SPACE و نترك أول
        // عنصر بالمصفوفة فارغاً كذلك
        char []arr = {' ','A','C','B','F','G',' ','D'};
        // أنشأنا شجرة بناء على هذه المصفوفة
        BinaryTree bt = new BinaryTree(arr);
        //preorder traversal starting from the root
        bt.preOrderTraversal(1);
        System.out.println();
        //inorder traversal starting from the root
        bt.inOrderTraversal(1);
        System.out.println();
        //postorder traversal starting from the root
        bt.postOrderTraversal(1);
        System.out.println();
    }
}

```



ملاحظة ارييسيزية:

سنتعرف على آخر نوع من الـ traversal وهو BFS و يتم أيضاً تسميته level order traversal .
 طبعاً لم نشرحه مع الـ DFS بصورة عامة و سبب ذلك يرجع إلى أنه لو أردنا تنفيذ الـ DFS بشكل تكراري يمكن ذلك باستخدام الـ stack و كون العودية داخلياً يعتمد على بنية تسمى call stack يجعل التعقيد الزمني لحد ما متماثلاً بين لو نفذناه تكرارياً أو عودياً فاختارنا الطريق الأسهل للفهم
 بينما لو كنا نتحدث عن BFS فإنه تكرارياً يعتمد على استخدام queue و هذا لا يماثل آلية عمل الـ ريكورجن و بالتالي حتى و إن كنا قادرين على عمل طريقة "عودية" لتنفيذ الـ BFS ستكون ذات تعقيد أعلى.

- $O(n \log n)$ في حالة كانت الشجرة متوازنة و العقد موزعة.
- $O(n^2)$ في حال كانت كل عقد الشجرة بجهة واحدة.

حيث n هو عدد العقد ، أما بالطريقة التكرارية يكون التعقيد $O(n)$ و لتنفيذه نقوم بالتالي:

```
public void levelOrderTraversal(char[] arr){
    //queue يتتبع الأبناء بالترتيب بناء على المستوى الذي يقعون به
    // بحيث كل أفراد المستوى الأول سيتم طباعتهم قبل أفراد المستوى الثاني و هكذا حتى تنتهي العقد كافة
    Queue<Integer> queue = new LinkedList<>();
    // نضيف الجذر إلى الطابور (نبدأ من الفهرس 1 لأن أول عنصر بالمصفوفة فارغ)
    queue.offer(1);
    // طالما يوجد عقد ضمن الـ queue أكمل
    while (!queue.isEmpty()) {
        // نأخذ أول عنصر دخل الطابور
        int i = queue.poll();
        // لو وصلنا لمرحلة خرجنا خارج إطار المصفوفة أو وصلنا لعقدة فارغة فلا تفعل شيئاً و ابحث في عنصر آخر
        if (i >= arr.length || arr[i] == ' ')
            continue;
        // لو كان العنصر يمثل عقدة فنطبعه
        System.out.print(arr[i]);
        // نضيف أبناء هذا العنصر إلى الطابور
        queue.add(2 * i);
        queue.add(2 * i + 1);
    }
}
```



لنتحدث الآن عن طريقة أخرى لتمثيل الشجرة و هي باستخدام البوينترز أو الـ

references

ببساطة نقوم بعمل كلاس لعقدة هذه العقدة فيها 3 متغيرات

المتغير الأول للبيانات التي تحملها هذه العقدة ، و متغيران يعبران عن مؤشرات لعقد الأبناء ، أحدهما للابن الأيمن و الآخر للابن الأيسر (يشبه الأمر تعاملنا مع الـ linked list لحد ما و لكن هذه المرة بدلاً من كون العنصر يحمل مؤشراً لعقدة واحدة فإنه هنا يحمل مؤشرين لعقدتين اثنتين).

سنقوم بعمل أنواع الـ traversal السابقة كذلك بطريقة التمثيل الجديدة

```
import java.util.Queue;

class BinaryTree{
    // جذر الشجرة
    Node root ;

    // كلاس داخلي لتمثيل العقد
    class Node {
        char value;
        Node left, right;

        // بالوضع الطبيعي عندما نعمل عقدة جديدة نسند لها قيمة و لا يكون لها أبناء
        Node(char value) {
            this.value = value;
            left = right = null;
        }
    }
}
```

// نفس المبدأ السابق تماماً لكن عوضاً عن الاستدعاء العودي على indexes سنقوم بالاستدعاء العودي على الأبناء كعقد فعلية و شرط التوقف يصبح أن تكون العقدة التي وصلنا لها null أما سابقاً فكنا نتفقد إن خرجنا خارج إطار المصفوفة أو وجدنا محرف space ضمن المصفوفة

```
public void preOrderTraversal(Node node) {
    if (node == null)
        return;
    System.out.print(node.value);
    preOrderTraversal(node.left);
    preOrderTraversal(node.right);
}
```

```

public void inOrderTraversal(Node node) {
    if (node == null)
        return;
    inOrderTraversal(node.left);
    System.out.print(node.value);
    inOrderTraversal(node.right);
}

public void postOrderTraversal(Node node) {
    if (node == null)
        return;
    postOrderTraversal(node.left);
    postOrderTraversal(node.right);
    System.out.print(node.value);
}

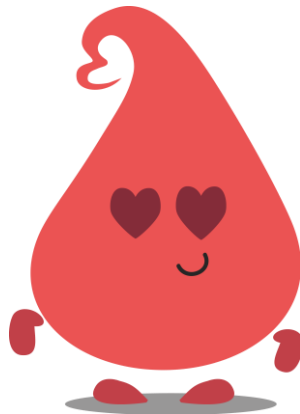
public void levelOrderTraversal() {
    if (root == null)
        return;
    // الـ queue يخزن عقداً بدلاً من indexes
    Queue<Node> queue = new LinkedList<>();
    queue.offer(root);
    while (!queue.isEmpty()) {
        Node current = queue.poll();
        System.out.print(current.value);
        if (current.left != null)
            queue.offer(current.left);
        if (current.right != null)
            queue.offer(current.right);
    }
}

```



```
public class Main {  
    public static void main(String[] args) {  
        // سنقوم بعمل ذات الشجرة التي استخدمناها سابقاً في المثال و سنقوم بذلك من خلال المؤشرات  
        BinaryTree bt = new BinaryTree();  
        bt.root = bt.new Node('A');  
        bt.root.left = bt.new Node('C');  
        bt.root.left.left = bt.new Node('F');  
        bt.root.left.right = bt.new Node('G');  
        bt.root.right = bt.new Node('B');  
        bt.root.right.right = bt.new Node('D');  
        // نستدعي التتابع على الجذر  
        bt.preOrderTraversal(bt.root);  
        System.out.println();  
        bt.inOrderTraversal(bt.root);  
        System.out.println();  
        bt.postOrderTraversal(bt.root);  
        System.out.println();  
        bt.levelOrderTraversal();  
    }  
}
```

كما نلاحظ استخدام العودية يسهل عملية ال DFS على المبرمج .



THE END